

Manual Técnico — CinemaUSAC Fase 2

1. Introducción

CinemaUSAC es una aplicación de escritorio desarrollada en **C++17** con **Qt 6** que administra la operación de un complejo de cines. Esta segunda fase amplía la Fase 1 eliminando la limitación de manejar una sola función, incorporando múltiples estructuras dinámicas:

- **Árbol Binario de Búsqueda (BST)** para la cartelera de películas.
- **Árbol AVL** para las funciones, indexadas por `codigo_funcion`.
- **Árbol B de orden 4** para los clientes, indexados por `id`.
- **Tabla Hash general** para las reservas, indexadas por `codigo_reserva`.
- **Matriz Dispersa** para la ocupación de asientos de la función activa.

Toda la persistencia se realiza con archivos **JSON** leídos y escritos por un **parser manual** desarrollado por los estudiantes (sin librerías externas).

2. Arquitectura del proyecto

```
CinemaUsac/
├── CinemaUsac.pro           # Configuración de qmake
├── main.cpp                 # Punto de entrada
├── InterfazGrafica/
│   ├── mainwindow.h / .cpp  # Ventana principal y toda la UI
│   └── PerfilDialog.h / .cpp # Diálogo para editar perfil de cliente
├── Modelos/
│   ├── Pelicula.h           # Modelo de película
│   ├── Funcion.h            # Modelo de función
│   ├── Cliente.h            # Modelo de cliente
│   ├── Reserva.h            # Modelo de reserva
│   ├── Promocion.h          # Modelo de promoción (Fase 1)
│   ├── Solicitud.h          # Modelo de solicitud (Fase 1)
│   └── Json.h / .cpp         # Parser y escritor JSON manual
├── Utilidades/
│   └── Validaciones.h       # Validación de fechas, correos, enteros, etc.
├── Estructuras/
│   ├── NodoArbol.h + ArbolPeliculas.h/.cpp # BST
│   ├── NodoAVL.h + ArbolAVL.h/.cpp         # Árbol AVL
│   ├── NodoB.h + ArbolB.h/.cpp             # Árbol B orden 4
│   ├── NodoHash.h + TablaHash.h/.cpp       # Tabla Hash
│   ├── NodoMatriz.h + MatrizAsientos.h/.cpp # Matriz Dispersa
│   ├── NodoListaDoble.h + ListaCircularDoble.h/.cpp # Solicitudes
│   └── NodoListaListas.h + ListaDeListas.h/.cpp # Promociones
├── Persistencia/
│   └── Persistencia.h / .cpp               # Carga/guardado JSON
├── Reportes/
│   └── GeneradorDot.h / .cpp               # Generación de reportes Graphviz
├── datos/                                # Archivos JSON persistentes
│   ├── cartelera.json
│   ├── clientes.json
│   ├── reservas.json
│   ├── F001_funcion.json
│   └── ...
└── documentación/                       # Este directorio
```

3. Estructuras de datos implementadas

3.1 Árbol Binario de Búsqueda (BST) — Películas

Archivos: `Estructuras/NodoArbol.h` , `Estructuras/ArbolPeliculas.h/.cpp`

- Clave de ordenamiento: `codigo` entero (derivado del `codigoOriginal` tipo `P001`).

- **Operaciones:** `insertar`, `eliminar`, `buscar`, `mostrarInorden`, `getRaiz`.
- **Recorridos soportados:** inorden, preorden y postorden (el selector está en la interfaz).
- **Colores en el reporte Graphviz:**
 - Verde: en cartelera.
 - Amarillo: próximo a retirar (menos de 7 días para `fecha_fin`).
 - Gris: fuera de cartelera o próximo estreno.

El BST no se modificó internamente respecto a la Fase 1; únicamente se agregaron los recorridos preorden y postorden y se integró con el nuevo sistema de carga masiva JSON.

3.2 Árbol AVL — Funciones

Archivos: `Estructuras/NodoAVL.h`, `Estructuras/ArbolAVL.h/.cpp`

- **Clave:** `codigo_funcion` (string, ej. `F001`).
- **Factor de balance:** `altura(izq) - altura(der)`.
- **Rotaciones implementadas:** LL (derecha), RR (izquierda), LR, RL.
- **Operaciones:** `insertar`, `eliminar`, `buscar`, `preorden`, `inorden`, `postorden`.
- **Justificación:** las funciones se crean y eliminan con frecuencia impredecible; el AVL garantiza búsqueda $O(\log n)$ incluso con muchas operaciones.

Detalle clave de la inserción:

```
actualizarAltura(nodo);
int b = balance(nodo);
if (b > 1 && f.codigoFuncion < nodo->izquierdo->funcion.codigoFuncion)
    return rotarDerecha(nodo);
if (b < -1 && f.codigoFuncion > nodo->derecho->funcion.codigoFuncion)
    return rotarIzquierda(nodo);
// ... rotaciones dobles
```

3.3 Árbol B de orden 4 — Clientes

Archivos: `Estructuras/NodoB.h`, `Estructuras/ArbolB.h/.cpp`

- **Orden:** 4 → máximo 3 claves por nodo, mínimo 1 (`ceil(4/2) - 1`).
- **Clave:** `id` del cliente (string, ej. `u001`).
- **Operaciones:** `insertar` (con split preventivo), `eliminar` (con redistribución y fusión), `buscar`, `listar`.
- **Eliminación:** se implementaron los cuatro casos clásicos:
 - La clave está en una hoja → eliminar directamente.
 - La clave está en un nodo interno y su hijo izquierdo tiene $\geq t$ claves → reemplazar por el predecesor.
 - La clave está en un nodo interno y su hijo derecho tiene $\geq t$ claves → reemplazar por el sucesor.
 - Ambos hijos tienen $t-1$ claves → fusionar y recursar.

3.4 Tabla Hash — Reservas

Archivos: `Estructuras/NodoHash.h`, `Estructuras/TablaHash.h/.cpp`

- **Tamaño:** 53 (primo, para reducir colisiones).

- **Función hash:** djb2 sobre `codigo_reserva`.
- **Colisiones:** resueltas por **encadenamiento** (listas enlazadas simples).
- **Operaciones:** `insertar`, `eliminar`, `buscar`, `listar`, `listarPorFuncion`, `listarPorCliente`.
- **Estadísticas expuestas al reporte:** tamaño, total de reservas, buckets ocupados y número de colisiones.

```
int TablaHash::funcionHash(const std::string& clave) {
    unsigned long hash = 5381;
    for (char c : clave)
        hash = ((hash << 5) + hash) + static_cast<unsigned char>(c);
    return static_cast<int>(hash % TAM_HASH);
}
```

3.5 Matriz Dispersa — Asientos

Archivos: `Estructuras/NodoMatriz.h`, `Estructuras/MatrizAsientos.h/.cpp`

- Representa la **función activa** en memoria.
- Solo almacena nodos para los asientos **ocupados**; los libres se representan implícitamente.
- **Cambio respecto a Fase 1:** `NodoMatriz::cliente` → `NodoMatriz::codigoReserva`. Cualquier otro dato de la reserva o del cliente se consulta en la **Tabla Hash** siguiendo ese código.
- **Ciclo de vida:**
 - Al seleccionar una función, si había otra cargada, se persiste su `.json` y se vacía la matriz.
 - Se carga el archivo `F00X_funcion.json` y se insertan los asientos ocupados.
 - Al reservar/cancelar, se actualiza la matriz y se persiste de vuelta.
- **Métodos nuevos:** `vaciar()`, `setCodigoFuncion()`, `getCodigoFuncion()`, `obtenerReservas()`, `buscarPorCodigoReserva()`.

3.6 Listas auxiliares (Fase 1, sin cambios)

- **Lista Circular Doblemente Enlazada** para solicitudes especiales.
- **Lista de Listas** (circular) para promociones con sublista de beneficios.

4. Validaciones

Archivo: `Utilidades/Validaciones.h`

Función	Verifica
<code>esFechaValida</code>	AAAA-MM-DD con días reales (contempla bisiestos)
<code>esEnteroPositivo</code>	Solo dígitos, valor > 0
<code>esEntero</code>	Dígitos con signo opcional
<code>esHorarioValido</code>	HH:MM entre 00:00 y 23:59
<code>esCorreoValido</code>	Contiene <code>@</code> y al menos un <code>.</code> después
<code>esTelefonoValido</code>	Solo dígitos, longitud 8-15

Las validaciones se aplican tanto a la entrada del usuario (formularios) como a la carga de archivos JSON. Los registros inválidos se **rechazan y reportan** sin afectar a los válidos.

5. Reportes Graphviz

Archivo: `Reportes/GeneradorDot.h/.cpp`

Reporte	Método	Color/Forma
BST Películas	<code>graficarArbol</code>	Cajas redondeadas; verde/amarillo/gris según estado
AVL Funciones	<code>graficarAVL</code>	Círculos; verde=programada, rojo=realizada
Árbol B Clientes	<code>graficarArbolB</code>	Cajas <code>record</code> divididas; azul=raíz, amarillo=interno, verde=hoja
Tabla Hash Reservas	<code>graficarHash</code>	Buckets numerados; azul=con datos, gris=vacío; colisiones resaltadas
Matriz Dispersa	<code>graficarMatriz</code>	Cabeceras rectangulares, asientos circulares (rojo=ocupado, libre=punteado)
Lista Circular Doble	<code>graficarListaDoble</code>	Elipses con flechas bidireccionales
Lista de Listas	<code>graficarListaListas</code>	Promociones en ciclo + sublista de beneficios

Todos los métodos generan un `.dot` en `reportes_generados/` y llaman a `dot -Tpng` para producir el PNG. El método `ejecutarGraphviz(dot, png)` centraliza la llamada a `system()`.

6. Compilación y despliegue

Requisitos

- Qt 6.2+ (probado con Qt 6.11).
- Compilador C++17 (GCC, Clang o MSVC).
- Graphviz instalado con `dot` accesible desde `PATH`.

Pasos

```
git clone <repo> EDD_2S2026_PY_<carnet>
cd EDD_2S2026_PY_<carnet>/EDD_Fase2
qmake CinemaUsac.pro
make          # o mingw32-make en Windows
./CinemaUsac
```

En Qt Creator:

1. Abrir `CinemaUsac.pro` .
 2. **Run qmake** (clic derecho sobre el proyecto).
 3. Compilar con Ctrl+B.
 4. Ejecutar con Ctrl+R.
-